

Meeting Understanding — the Scribe and the live Meeting Page

How Proxy turns live speech into one always-current, cited, machine-readable page. Part A is the product — what the page is and every capability it unlocks. Parts B–C are the build — the pipeline in one picture, then component by component: what each piece is, how we wire it in, and how we optimize it, specific enough to implement.

Deep Dive 01 built a map of the company's software. This one builds the map of *the meeting* — live, as people talk. By the time the words leave someone's mouth, Proxy has already turned them into a structured, cited understanding that every other part of the system reads. This is the layer that makes a meeting *fully understood*, in real time, for any kind of meeting.

THE ONE IDEA TO HOLD

One accountable agent — **the Scribe** — continuously writes **one living document, the Meeting Page**: the meeting's understanding, exactly what the best chief of staff would keep if they could type as fast as people talk. Under the Scribe runs a **fast reflex pipeline** (cheap models and free gates on the hot path) that does the routine second-by-second writing; the Scribe is the *intelligence* that steps in exactly where the reflexes hit ambiguity. The page is not a transcript and not a summary — it is a **session graph**: claims, decisions, referents, and open questions, each pointing at the real thing it is about. Everything below is how speech becomes that page, cheaply and without ever guessing silently.

PART A — THE PRODUCT

What the Meeting Page is, and everything it makes possible

The capabilities a product developer needs to know — written before any implementation detail.

The Meeting Page is a single, always-current document that *is* the meeting's understanding. Where a transcript is a wall of words and a summary is a stale afterthought, the page is live, structured, and cited: it holds what is true right now, who said it, when, and where it can be checked. It is the

working memory of the whole system — the one surface every other part of Proxy reads from and reacts to.

What is on the page

The page carries the meeting's whole state, in machine-exact structure where it must be exact and in plain prose where it must convey meaning.

- **The goal, and how it's going.** The goal of the meeting, the distance still to cover, and the momentum — *converging*, *circling*, or *stalled* — plus the single **current blocker**: what is most in the way of that goal right now.
- **Claims.** Every checkable thing anyone asserts — the exact value, who said it, the timestamp — for example "checkout's error rate is 2% (Priya, 10:12)."
- **Decisions.** The decision currently forming, its status, how reversible it is, where each person stands, and the **blast-set** — which parts of the software it would touch.
- **Referents.** What the pronouns and shorthand actually mean — "that migration" resolved to the real pull request, "the dashboard" to the real service — each linked to its page in the Estate Graph from Deep Dive 01.
- **Open questions and commitments.** The questions nobody has answered yet, and who owes what follow-up.
- **Unresolved debate, first-class.** A disagreement with no consensus is not dropped — it is held as proposals, arguments, stances, and an **OPEN** status, so a live argument is captured honestly rather than flattened into a fake conclusion.

It records **observable dynamics only** — who is driving, who owns a topic but hasn't spoken, that a disagreement is unresolved — and **never inferred emotion**. It reports what happened in the room, not what it imagines people feel.

The capabilities this unlocks

Because the page is structured and cited rather than a blob of text, Proxy can do things a meeting summarizer never could:

- **Catch a contradiction across time.** A number stated at minute three and quietly contradicted at minute twenty is catchable *from the page alone* — because both land as claims resolving to the same thing, and the mismatch is visible as structure (a **tension edge**, below), not something a model has to happen to remember.

- **Answer "is that true?" instantly.** Every checkable claim carries a **check-pointer** — the exact estate node and live source where it would be verified — so the answer is one lookup away, with no scramble to figure out what the question is even about.
- **Know what's fresh.** Each claim about a live system carries a **freshness stamp** — how long ago Proxy last verified that system — so it knows when a cached truth still stands and when a re-check is worth it.
- **Produce the post-meeting output for free.** The decisions, action items, and per-person follow-ups are *already on the page, already cited*. There is no separate "generate the summary" step; the final page state is the output.
- **Let anyone correct it live.** Because it's a readable document, the companion panel shows it forming as living meeting notes, and any participant can fix a mis-captured decision or rename a referent — the correction re-enters the same writing process and takes effect immediately.

WHY THE PAGE IS WHAT WAKES THE WHOLE SYSTEM

The Meeting Page is not just a record for humans — it is the **nervous system** of the entire product. Every write the Scribe makes is an *event* other agents react to. Resolving "checkout" to its estate node is exactly what summons checkout's expert to help. A claim appearing wakes the claim-checker; a decision moving toward final wakes the decision-watcher. The rest of Proxy doesn't poll or re-read the transcript — it watches this one page change. Get the page right and everything downstream has what it needs; that's why this layer is built first and held to the highest bar.

It works for any meeting, with nothing branching on type

A standup, a production incident, a design review, a sales call, and a board meeting all run the *identical* mechanism. Nothing on the page or in its writer asks "what kind of meeting is this?" What differs is only which content shows up, and the schema is a **type-superset** that absorbs that difference by content, not by branching code:

- **Event / timeline nodes** appear when the meeting is an incident ("at 10:04 the alert fired; at 10:09 we rolled back") — a timeline builds itself from the same writer.
- **Metric nodes** appear when numbers are on the table — a quarterly business review fills with revenue, churn, and error-rate values pinned to their sources.
- **Topic / Section regions** appear in long meetings — the graph is shaped like the meeting itself, topics are *regions* of it, and a re-opened region is the signal that the room is circling back on something.

One writer, one code path, instantiated by whatever the room actually talks about. There is no per-vertical logic anywhere — a sales call and a board review differ only in their content, never in the machine.

It is enterprise-size- and vertical-agnostic

Nothing in this layer knows or cares how big the company is or what industry it's in. The page tracks a two-person standup and a forty-person all-hands with the same fields, because it holds the *meeting's* state, which is small regardless of company size. And because cost is tied to the page (which stays small) rather than the transcript (which grows), a three-hour meeting costs almost the same to understand as a thirty-minute one. The industry never enters: the page holds claims and decisions whether they're about payment services, clinical pipelines, or game-engine modules.

PART B — HOW IT WORKS

From speech to page, end to end

The whole pipeline in one picture, then in words.

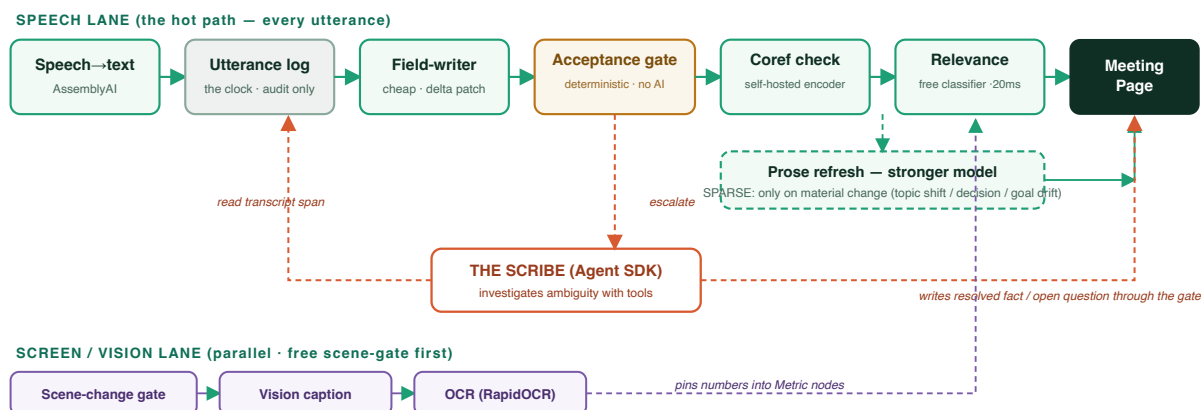


Figure 1 — The full pipeline. The green speech lane is the hot path: it runs on every utterance and is deliberately mechanical. The orange path is the Scribe — the one intelligence, invoked only where the reflexes hit ambiguity. The purple lane watches the screen in parallel. Everything writes to one page, through one gate.

The flow in words

Read the green lane left to right. **Speech-to-text** turns the audio into text utterances as people talk. Each utterance is appended to the **utterance log** — an append-only record that serves as the meeting's single clock and its audit trail, but which *nothing reads live* (its only jobs are replay and stamping versions in time). On each new utterance, a cheap **field-writer** model proposes a small **delta patch** to the page — only the fields that changed. That patch passes through a **deterministic acceptance gate** — plain code, no AI — that rejects anything malformed. An **independent coreference check** on a specialized self-hosted model verifies that "it"/"that" resolved to the right thing; a below-floor referent is written as a *question*, never asserted. A free **relevance classifier** drops chit-chat so only substance lands. The result updates the **Meeting Page**. Sparsely — only when something material changes — a **stronger model** refreshes the page's prose read of "what's really going on here."

Two things about this flow are the whole design. First, **the Scribe is the intelligence and the reflexes are the hot path**. The green lane is a fast mechanical pipeline because — as three research passes confirmed — pipelines beat agent loops on latency, cost, and determinism for the routine per-utterance work. But responsibility for the page belongs to *one real agent*. Whenever the reflexes hit

ambiguity — a referent below the confidence floor, two claims that contradict, garbled audio, an entity nobody's heard of — they **escalate to the Scribe** (the orange path), which investigates with real tools and resolves it, rather than letting a low-confidence guess land. Second, **the page is the only thing anyone reads**. Every downstream consumer reads the small page, never the growing transcript — which is why the cost stays flat no matter how long the meeting runs.

THE PAGE IS A SESSION GRAPH — THE SCHEMA, IN PLAIN TERMS

The Meeting Page is stored as a graph, the same shape as the Estate Graph from Deep Dive 01: **nodes** and typed **edges**. A **claim** node holds a value, a speaker, and a timestamp, plus a *check-pointer* (the estate node and live source where it verifies) and a *freshness stamp*. A **decision** node holds a status, a reversibility, each person's stance, and a blast-set. A **referent** is resolved to an Estate Graph node. There are **commitments**, **open questions**, and the **goal** with its distance and momentum and its *current blocker*. Node types are a superset — **Event** (incident timelines), **Metric** (numbers), **Topic/Section** (regions of a long meeting; a re-opened region is the circling signal). A **tension edge** is written whenever two claims — or a claim and a live estate value — resolve to the *same node with incompatible values*: a pre-contradiction, visible before either speaker notices. Unresolved debate is a first-class subgraph: proposals, arguments, stances, and an **OPEN** status.

PART C — THE BUILD

Every piece, exactly — what it is, how we wire it, how we optimize it

For each component: first the overview (what it is and why it's here, in plain terms), then the technical specifics (how we integrate and tune it). Dense, but built to be understood.

Component 1 — Speech-to-text: the raw input

Overview. Everything starts with turning audio into text, fast and accurately, with each utterance tagged by who said it. Bad transcription poisons everything downstream, so this is the one place we default to a strong paid model — but we build a cheap-flag so a cost-sensitive customer can dial it down.

Technical specifics — how we wire and optimize it. The default is **AssemblyAI Universal-3 Pro** for its accuracy and speaker labeling ("diarization" — knowing which voice is which). It streams partial results so the page can update within a couple of seconds of speech. Every utterance event carries two extra signals we compute from the audio: an *input-quality* stamp (was this clean or garbled?) and an *interpretation-confidence* stamp — these ride downstream so the gate and the Scribe know how much to trust each word. We optimize cost with a **cheap-flag**: because the vendor sits behind a swap adapter (the same discipline as everywhere in Proxy), a customer who wants a lower bill can route to a cheaper open-weight recognizer, and nothing else in the pipeline changes. Speech-to-text is ~60% of a meeting's plumbing cost, so this dial is the main lever for the cost envelope without touching the judgment core.

Component 2 — The utterance log: the single clock

Overview. Every utterance is appended to a log that is never edited and never deleted. This is *not* what the system reads to understand the meeting — that's the page. The log's only jobs are replay/audit and, more importantly, being the meeting's single source of time.

Technical specifics. It's an append-only table in Postgres, one row per utterance, monotonically numbered. This is a **consistency invariant**, not just a record: every version of the Meeting Page is stamped to the utterance that produced it. That gives the whole system one unambiguous timeline — so any agent reasoning over the page knows exactly which state it saw, and a draft written against minute-3's page is *detectably stale* at minute-20 and discarded rather than silently acted on.

We optimize by keeping the log entirely off the hot path: because nothing reads it live, it can be written lazily and never sits between speech and the page.

Component 3 — The field-writer: cheap, per-utterance, delta-only

Overview. This is the workhorse that runs on *every single utterance*. It reads the new words and the current page and proposes the smallest change: "add a claim," "move this decision to forming," "resolve 'that migration' to PR #482." Because it runs constantly, it must be cheap — so it's a small fast model, and it only ever emits a delta, never a rewrite.

Technical specifics — how we optimize it. It runs on a **Haiku-class** (small, fast, cheap) model on the Agent SDK. Two optimizations keep it flat and cheap: it emits a *patch* (a list of field changes) not a whole page, so its output is tiny; and it reads the *small structured page*, never the transcript, so its input never grows with meeting length. We **coalesce** rapid-fire utterances — several short back-to-backs from one speaker are batched into one call rather than paying per fragment. Its prompt is not hand-authored forever: it's a target of the **fine-tune flywheel** (Component 9), so over weeks it's replaced by a version tuned on this org's own accepted and rejected patches — several-fold cheaper and more accurate.

SYSTEM PROMPT — FIELD-WRITER (ILLUSTRATIVE)

You maintain THE MEETING PAGE. Input: the new utterance(s) + the current page (structured fields only – never the transcript). Output: the SMALLEST patch.

Emit **ONLY** changed fields, as operations:

```
add_claim(value, speaker, ts)      set_decision_status(id, forming|final)
resolve_referent(text, node_hint)  add_open_question(text)
add_stance(decision_id, person, position)  note_commitment(person, what)
```

Hard rules:

- Every claim carries speaker + timestamp. Never invent a node id or a referent.
 - Never revert a decision's status on your own – only forward, and only on clear spoken evidence.
 - If a referent is unclear, DO NOT guess – emit `resolve_referent(text, UNSURE)`; the coref model and the Scribe decide. A wrong "it" is the one poison that lasts.
 - Say nothing about emotion. Observable dynamics only.
- Return an empty patch if nothing material changed.

Component 4 — The acceptance gate: deterministic, no AI

Overview. Nothing the field-writer proposes touches the page until it passes a gate made of plain code — no model, so it's free, instant, and perfectly predictable. The gate is what lets us trust a

cheap model to maintain the page: the model can be sloppy because the gate is strict.

Technical specifics — the exact checks. Every patch must be (1) **schema-valid** — the right fields, right types; (2) free of **invented IDs** — a referent may only resolve to a real Estate Graph node; (3) **pointer-sound** — anything the patch references must already exist on the page; and (4) **status-monotonic** — a decision's status cannot silently revert (a decision can't slide from "final" back to "forming" without explicit spoken evidence). A patch that fails is rejected and the ambiguity that caused it is escalated to the Scribe rather than dropped. This gate is also the **single-writer seam**: participant corrections and Proxy's own contributions never write the page directly — they enter through this same gate — so there is never a write-write conflict on the meeting's understanding. One hand holds the pen; everyone else reads.

Component 5 — The coreference resolver: a specialist model, because "it" is the poison

Overview. Coreference is figuring out what "it," "that," "the migration," "the old one" point to. Getting this wrong is the single most dangerous failure in the whole layer: a confidently mis-resolved "it" produces a claim that *looks* grounded and cited and sails through every downstream check — it is the one poison that survives verification. So we treat it as its own dedicated component, not a side job of the writer.

Technical specifics — how we optimize it. We run a **dedicated, self-hosted coreference encoder model** — a specialized model, not an LLM — because specialized encoders measurably beat general LLMs at exactly this task (on the order of ~13 F1 points; "F1" is the standard accuracy score for this kind of task, higher is better). It's self-hosted, so it's ~free per call and runs in milliseconds on the hot path. It cross-checks the field-writer's referent resolution independently. The rule that makes it safe: **anything below the confidence floor is written as an open question, never as an asserted fact** — the page will say "'it' here is unclear — the migration, or the rollback?" instead of silently picking one. Below-floor cases escalate to the Scribe, which can read more context and resolve them properly. We never let a low-confidence referent land as a bare fact.

Component 6 — The relevance classifier: free, 20ms, drops the noise

Overview. Most of what's said in a meeting isn't page-worthy — greetings, tangents, "can you hear me?" A tiny classifier decides whether an utterance carries substance worth writing, so the page stays a clean signal and the more expensive stages downstream never fire on noise.

Technical specifics — how we optimize it. This is a **self-hosted classifier replicating a published production recipe** — roughly **F1 0.895**, about **20ms** per call, and effectively **free** because it runs

on our own hardware with no per-call model fee. It is, per the master, the cheapest quality lever in the whole layer: a near-free gate that keeps every downstream cost proportional to *substance* rather than to talk. Because it's self-hosted and tiny, it adds negligible latency to the hot path while cutting the volume that reaches the writer's more expensive judgment.

Component 7 — The prose refresh: a stronger model, but sparse

Overview. The structured fields capture *what* is true; a short block of prose captures *what's really going on here* — the read a smart chief of staff would give. That read needs a stronger model, so we do it rarely: only when something material changes.

Technical specifics — how we optimize it, and what it writes. A Sonnet/Opus-class model refreshes the prose read *only on material change* — a topic shift, a decision moving, goal drift — a handful of times per meeting segment, cents not dollars, never per utterance. This is the core cost optimization of the layer: **cheap model on every utterance, strong model only on the sparse refresh**. The refresh also writes the *derived forward-looking signals* the master specifies, all computed mechanically from structure the pass already touches: the **current blocker; trajectory and time pressure** (what's likely to be decided and when, from agenda-remaining versus clock and decision velocity); and it triggers the **tension-edge** and **stakes** annotations. None of this migrates judgment into the Scribe — it just writes down more of what the passes already compute, making the page *addressable* so downstream agents reason over structure instead of doing "prose archaeology."

Component 8 — The Scribe: one accountable agent over the reflexes

Overview. The Scribe is the *one agent* that owns the page. The reflex pipeline (Components 3–7) does the fast routine work; the Scribe is the intelligence that steps in exactly where the reflexes hit something they can't safely handle. It turns a would-be silent error into either a resolved fact or an honest open question — never a low-confidence guess.

Technical specifics — what it is and its tools. The Scribe is a **Claude Agent SDK** session — a think → call-a-tool → read-result → repeat loop. It's invoked on **escalation**: a sub-floor referent, a claim contradiction, garbled input, or an unknown entity. Its tools let it actually investigate: *read the transcript span* (the one place the raw log is read, and only on demand), *query the Estate Graph*, *re-run the coreference resolver with more context*, *check the screen lane*, *edit the page through the gate*, *write an open question*, or *fire a quiet clarifying DM* to a participant. The sparse prose refresh is the same agent's periodic deliberate read of the room. Escalations are rare — a handful per meeting, cents — which is exactly why we can afford real intelligence there. We optimize by keeping the

Scribe *off* the per-utterance path: it costs money only when ambiguity genuinely needs it, so the hot path stays cheap and the intelligence sits precisely where it earns its keep.

SYSTEM PROMPT — THE SCRIBE (ILLUSTRATIVE)

You are the Scribe: the one agent accountable for THE MEETING PAGE. A fast reflex pipeline maintains the page under you. You are invoked when it ESCALATES – a referent below the confidence floor, a contradiction between two claims, garbled input, or an entity no one has mapped.

Your job: resolve the ambiguity into a grounded fact, or an honest open question. Never let a low-confidence guess reach the page.

Tools:

```
read_transcript_span(from_ts, to_ts)    query_estate_graph(node|query)
rerun_coref(referent, extra_context)    check_screen_lane(near_ts)
edit_page(patch)    # goes through the same acceptance gate
write_open_question(text)    clarifying_dm(person, question)
```

Procedure:

- Investigate before you write. Read the span, check the graph, re-run coref.
 - If you can ground it: `edit_page` with the resolved fact + its check-pointer.
 - If two claims genuinely conflict: leave the tension edge; do not pick a winner.
 - If you still cannot resolve it: `write_open_question`, or DM the one person who would know. Abstain rather than assert. Silence is a valid, honest output.
- Every write is stamped to the current utterance (the one clock).

Component 9 — The screen / vision lane: what is shown, pinned exactly

Overview. A lot of a meeting lives on the shared screen — a dashboard, a slide, a diagram. This parallel lane watches the screen and pins exact on-screen facts (especially numbers) onto the page, without letting a scrolling demo blow the budget.

Technical specifics — three cheap stages. First a **free scene-change gate** compares frames and keeps only the ones where the screen *truly* changed — roughly **30–80 frames an hour**, and **hard-capped** so a demo, a screen-share scroll, or a video can never explode the count. Then a **small vision model** captions each kept frame ("a Grafana dashboard showing checkout error rate at 3.4%"). Then **free local OCR (RapidOCR)** reads exact on-screen text and pins the numbers into **Metric nodes** on the page — so "the dashboard says 3.4%" becomes a real, checkable claim rather than a fuzzy memory. We optimize by making the free scene-gate do the filtering: the paid vision model only ever sees the handful of genuinely-new frames, and the hard cap guarantees the lane's cost is bounded regardless of what's on screen.

Component 10 — The self-improvement flywheel: corrections are free training data

Overview. Every time a human fixes the page, or the Scribe resolves an escalation, or a write is accepted or rejected, that is a labeled example of the *right* answer. We log all of it. Later, that log trains a small model tuned to this exact org.

Technical specifics — how we wire it. This is a **day-one schema decision**, not a later feature: every correction, escalation resolution, and accept/reject verdict is written to a shadow ledger as labeled training data — the same ledger the whole society writes to. That corpus feeds **small-model fine-tuning** that replaces the stock cheap field-writer with a version tuned on the org's own examples, making the Scribe **several-fold cheaper and more accurate per org** over weeks. The optimization insight is that *catching the corrections is nearly free* — the humans are already correcting the page for their own benefit; we pay one logging decision now and harvest a compounding accuracy-and-cost win later. The corrections are free exhaust.

How downstream consumes the page — the load-bearing seams

Overview. The page isn't built for humans to read (though they can) — it's built for the rest of Proxy to react to. Three seams are load-bearing, and each is an *optimization* that makes the rest of the system cheap.

- **Constant-time reads.** Every consumer reads the small structured page, never the growing transcript — so a two-hour meeting is no more expensive to reason over than a five-minute one. This is *the* reason cost stays flat with meeting length.
- **Each resolved referent benches an expert.** When the Scribe resolves "checkout" to its estate node, that resolution *summons* checkout's expert (from Deep Dive 01) to the live bench and stands it down when the topic leaves. The page's links *are* the routing — there's no separate router, no classifier, no LLM call to decide who handles what.
- **Each check-pointer's freshness stamp drives the tier decision.** A claim stamped static-and-valid routes a later "is that true?" to a cache read; a versioned-or-expired claim routes it to a live lookup. The annotation the Scribe writes here is exactly what lets the action layer's ~10ms router decide how much work a question needs, with no model call.

Merge-back at close — the one deliberate reconciliation

Overview. When the meeting ends, everything it learned that belongs in the company's long-term memory folds back into the Estate Graph — so next week's standup starts already knowing today's

decisions.

Technical specifics. The page's final state is merged down onto the affected estate pages, with the meeting as the source. On conflict with an existing estate claim we **supersede-with-timestamp and route to re-verify** — never a silent overwrite; both truths are kept, the new one stamped, and Deep Dive 01's re-verification runs. Recurring meetings **pre-seed from their predecessor's open tail** via a `continues` edge, so a meeting series becomes a queryable thread and the open questions carry forward. This is the one place the two memories deliberately reconcile; everywhere else the single-writer page, the one clock, and the per-row ledger keep the society coherent in real time.

How the layer stays cheap, fast, and length-proof

- **Cheap-on-the-hot-path, strong-on-the-sparse-path.** The per-utterance work runs on a small model behind free gates; the expensive stronger model fires only on the sparse prose refresh, cents per segment. The judgment stays frontier-grade exactly where it's rare enough to afford.
- **Free self-hosted models for the two specialist jobs.** Coreference (a specialized encoder) and relevance ($\sim$ F1 0.895, $\sim$ 20ms) both run on our own hardware at $\sim$ zero per-call cost — the cheapest quality levers in the layer.
- **Cost is flat in meeting length.** Because every consumer reads the small page and never the transcript, and the utterance log stays off the read path, a three-hour meeting costs essentially what a thirty-minute one does. Cost tracks the *size of the understanding*, which is bounded, not the volume of words.
- **Coalescing and hard caps bound the bursts.** Rapid utterances are batched into one field-writer call; the screen lane's scene-gate and hard frame cap keep vision cost bounded even through a scrolling demo or a played video.
- **The fine-tune flywheel drives marginal cost down over time.** Free correction-exhaust becomes labeled data becomes a tuned small writer — so the layer gets both cheaper and more accurate the longer an org uses it.
- **Vendors sit behind swap adapters.** Speech-to-text, vision, and OCR are all behind the same adapter discipline as everywhere in Proxy — a cheaper or better vendor is a one-adapter swap, and nothing else in the pipeline changes.

Done when

This layer is complete when: the Meeting Page stays current within a couple of seconds of live speech on a real multi-party meeting; referent resolution beats its accuracy floor on a labeled set and every sub-floor resolution is flagged as a question, never asserted; a claim made at minute three and contradicted at minute twenty is catchable from the page alone; the page's prose read demonstrably improves downstream answer quality on a replay set versus fields-alone; every consumer reads the page in constant time regardless of meeting length; every reflex escalation is

resolved by the Scribe or lands as an explicit open question, with no silent low-confidence writes; a participant can correct the page mid-meeting and the correction takes effect immediately through the same gate; the final page state produces the post-meeting outputs with no additional pass; the screen lane's frame cost stays bounded through a scrolling demo; the merge-back at close writes the meeting's decisions onto the right estate pages under supersede-with-timestamp; and nothing on the page or in its writer branches on meeting type — a standup, an incident, a sales call, and a board review run the identical mechanism.